

목차

1. implementation goal
2. Scheduler implementation
3. system call implement
4. implementation detail
5. results
6. conclusion
7. appendix

구현 목표

1. FCFS 스케줄러 작성
 - A. 먼저 생성된 process가 먼저 실행되어야 하고 끝날 때까지 또는 interrupt가 발생할 때까지 실행된다. (timer interrupt는 적용되지 않는다.)
2. MLFQ 스케줄러 작성
 - A. multi level feedback queue를 구현해야한다. level은 3이다.
 - B. level 0 > level 1 > level 2 순으로 가장 우선도가 높다.
 - C. level 0과 level 1은 round robin으로 동작한다.
 - D. level 2는 priority가 높은 원소가 우선된다.
 - E. i-level 큐의 원소는 $2^i + 1$ tick의 time quantum을 가지며 time quantum을 다 소진하면 아래 레벨 큐로 이동하고 time quantum이 초기화된다..
 - F. priority는 level 2에서 time quantum을 다 소모하면 1감소하고 0에서는 더 이상 감소하지 않는다.
 - G. 50tick이 되면 모든 process를 level 0큐에 다시 넣는 priority boosting을 구현해야 한다.
3. system call 구현
 - A. void yield()
 - i. CPU점유를 양보하는 system call
 - B. int fcfsmode()
 - i. schedule mode를 fcfs로 바꾼다. 이미 fcfs면 -1 아니면 0을 리턴한다.
 - C. int mlfqmode()
 - i. schedule mode를 mlfq로 바꾼다. 위와 비슷하게 동작한다.

- ii. priority는 3, level은 0으로 초기화한다.
- D. int getlev()
 - i. 현재 프로세스의 큐레벨을 반환한다. fcfsmode라면 99를 반환한다.
- E. int setpriority(int pid, int priority)
 - i. pid process의 우선도를 priority로 설정한다. priority가 0~3이 아니면 -2, pid가 존재하지 않으면 -1을 리턴한다.

Scheduler Design

- Queue design

모든 큐는 다음의 structure를 사용해 구현했다. QNUM은 queue하나가 가질 수 있는 최대 원소 수이다.

```
struct queue{
    struct spinlock lock;
    int s, e;
    struct proc *p[QNUM];
};
```

```
struct queue mlfq[3], fcfs;
```

L0, L1, FCFS 스케줄링에서 사용하는 큐는 원형 큐를 사용한다. 여기서 queue의 s는 head, e는 tail을 의미한다. 그래서 pop에서는 s에 해당하는 원소를 return하고 1증가하고 push할 때는 e에 process pointer를 추가하고 e를 1증가한다. +연산을 할 때 원형 큐에서는 원래 모듈러 연산을 사용하지만 큐의 크기를 2의 거듭제곱수로 설정하면 비용이 낮은 bitwise and를 사용할 수 있다.

```
void
queue_push(struct queue *q, struct proc *p)
{
    q->p[q->e] = p;
    q->e = (q->e + 1) & QMOD;
}
```

```
struct
proc *queue_pop(struct queue *q)
{
    struct proc *p = q->p[q->s];
    q->s = (q->s + 1) & QMOD;
    return p;
}
```

L2는 가장 큰 priority를 가진 원소를 반환해야 한다. 처음에는 max-heap으로 구현하려 했으나 setpriority함수 구현이 너무 복잡해져서 단순 배열로 구현했다. push연산은 배열의 뒤에 process pointer를 추가한다.

```
void
L2_push(struct queue *q, struct proc *p){
    q->p[q->e++] = p;
}
```

pop은 priority가 가장 크면서 배열의 앞부분에 있는 원소를 내보내게 했다. 또한 pop연산에서 가장 큰 원소를 찾은 후에는 그 위치부터 끝까지 한칸씩 앞으로 당겨

졌다. 이러한 구현은 heap으로 구현한 것과 달리 먼저 queue 들어온 원소가 나가기 쉽게 설계된다.

```
struct proc *
L2_pop(struct queue *q)
{
    struct proc *r = 0;
    int pr = -1;
    int idx = q->e;
    for(int i = 0; i < q->e; i++) {
        acquire(&q->p[i]->lock);
        if(pr < q->p[i]->priority) {
            r = q->p[i];
            pr = r->priority;
            idx = i;
        }
        release(&q->p[i]->lock);
    }
    q->e--;
    for(int i = idx; i < q->e; i++) my_swap(&q->p[i], &q->p[i+1]);
    return r;
}
```

- enqueue 동작

큐에 프로세스를 넣는 상황은 세 가지이다.

- RUNNING -> RUNNABLE
- SLEEPING -> RUNNABLE
- USED -> RUNNABLE

첫 번째 상황은 timer interrupt가 발생해서 CPU를 점유하고 있던 process가 다른 process에게 넘겨줘야 하는 것이다. 이는 scheduler 함수 내에서 process의 cpu점유가 끝나고 runnable 인지를 판단하여 queue에 넣는다.

```
void
my_switching(struct proc *p, struct cpu *c)
{
    p->state = RUNNING;
    c->proc = p;
    swtch(&c->context, &p->context);
    c->proc = 0;
    if(p->state == RUNNABLE) my_queueing(p);
    release(&p->lock);
    acquire(&priority_boost_lock);
    if(priority_boost) priority_boosting();
    priority_boost = 0;
    release(&priority_boost_lock);
}
```

cpu 점유 이후 c->proc = 0; 부터 실행한다.

두 번째 상황은 waiting 중에서 실행가능한 상태가 되는 것이다. 이는 wakeup함수와 kill 함수에서 발생한다.

```

int
kill(int pid)
{
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->pid == pid){
            p->killed = 1;
            if(p->state == SLEEPING){
                // Wake process from sleep().
                p->state = RUNNABLE;
                add_proc_to_queue(p);
            }
            release(&p->lock);
            return 0;
        }
        release(&p->lock);
    }
    return -1;
}

```

```

void
wakeup(void *chan)
{
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++) {
        if(p != myproc()){
            acquire(&p->lock);
            if(p->state == SLEEPING && p->chan == chan) {
                p->state = RUNNABLE;
                add_proc_to_queue(p);
            }
            release(&p->lock);
        }
    }
}

```

세번 째는 새로운 프로세스가 생성된 상황이다. userinit에서 초기 프로세스가 생성되고 fork함수에서 새로운 프로세스가 생성되는 상황이다.

```

fork(void)

    acquire(&np->lock);
    np->state = RUNNABLE;
    add_proc_to_queue(np);
    release(&np->lock);

    return pid;
}

```

```

userinit(void)

    p->state = RUNNABLE;
    add_proc_to_queue(p);

    release(&p->lock);
}

```

1번 상황은 time quantum을 1 감소 시키고 priority, queue-level을 time quantum=0 이라면 적절히 priority와 demotion을 해서 넣고 2, 3은 초기화하여 넣는다. time quantum은 clockintr함수에서 줄어들게 했다.

```

void
my_queueing(struct proc *p)
{
    acquire(&schedule_lock);
    if (schedule_mode == FCFS_MODE) push(&fcfs, p);
    else
    {
        if (p->time_quantum == 0)
        {
            if (p->level != 2)
                p->level++;
            else if (p->priority > 0)
                p->priority--;
            p->time_quantum = p->level * 2 + 1;
        }
        push(mlfq + p->level, p);
    }
    release(&schedule_lock);
}

```

```

void
add_proc_to_queue(struct proc *p)
{
    acquire(&schedule_lock);
    if (schedule_mode == FCFS_MODE) push(&fcfs, p);
    else
    {
        p->level = 0;
        p->priority = 3;
        p->time_quantum = p->level * 2 + 1;
        push(mlfq + p->level, p);
    }
    release(&schedule_lock);
}

```

- dequeue 동작

fcfs의 경우는 큐에서 RUNNABLE인 프로세스가 나올 때까지 pop한다. mlfq는 l0부터 l2까지 RUNNABLE인 프로세스가 나올 때까지 pop한다.

```

struct proc *
select_with_fcfs(void)
{
    while(!isEmpty(&fcfs)){
        struct proc *t = pop(&fcfs);
        acquire(&t->lock);
        if(t->state == RUNNABLE) return t;
        release(&t->lock);
    }
    return 0;
}

```

```

struct proc *
select_with_mlfq(void)
{
    for(int i = 0; i < 3; i++){
        while(!isEmpty(mlfq + i)){
            struct proc *t = pop(mlfq + i);
            acquire(&t->lock);
            if(t->state == RUNNABLE) {
                return t;
            }
            release(&t->lock);
        }
    }
    return 0;
}

```

- 스케줄링 동작

```
int schedule_mode = FCFS_MODE;
```

schedule_mode 변수를 사용해 FCFS와 MLFQ를 구분하여 동작한다. schedule 모드에 따라 queue에서 선택하는 방식을 달리한다.

```

void
my_scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    //w_sie(r_sie() & ~SIE_STIE);

    for(;;){
        intr_on();
        acquire(&schedule_lock);
        p = schedule_mode == FCFS_MODE ? select_with_fcfs() : select_with_mlfq();
        release(&schedule_lock);
        if(p) my_switching(p, c);
        else {
            intr_on();
            asm volatile("wfi");
        }
    }
}

```

FCFS에서는 timer interrupt가 발생해도 CPU 점유를 다른 process에게 넘기면 안된다. 따라서 schedule_mode 전역변수로 현재 mode가 FCFS인지를 체크하여 아니라면 CPU점유를 넘기고 맞으면 yield하지 않게 한다.

```

// give up the CPU if this is a timer interrupt.
acquire(&schedule_lock);
if(which_dev == 2 && schedule_mode != FCFS_MODE){
    release(&schedule_lock);
    yield();
}
else release(&schedule_lock);

```

usertrap(void)

```

// give up the CPU if this is a timer interrupt.
acquire(&schedule_lock);
if(which_dev == 2 && myproc() != 0 && schedule_mode != FCFS_MODE){
    release(&schedule_lock);
    yield();
}
else release(&schedule_lock);

```

kerneltrap(void)

- priority boosting

priority boosting 50 tick마다 발생한다. 따라서 tick이 증가하는 clockintr함수에서 MLFQ mode인 경우 priority boosting을 전역변수로 예약해두고 my_switching에서 이 변수를 확인하여 발생하게 했다. priority_boosting 구현은 단순히 모든 큐를 전부 0으로 초기화하고 모든 process들의 priority, level을 초기화한다. 그리고 level 0큐에 넣는다.

```
void
clockintr()
{
    if(cpuid() == 0){
        acquire(&tickslock);
        acquire(&schedule_lock);
        ticks++;
        if(ticks % 50 == 0 && schedule_mode == MLFQ_MODE) {
            acquire(&priority_boost_lock);
            priority_boost = 1;
            release(&priority_boost_lock);
        }
        wakeup(&ticks);
        release(&schedule_lock);
        release(&tickslock);
    }

    // ask for the next timer interrupt. this also clears
    // the interrupt request. 1000000 is about a tenth
    // of a second.
    w_stimecmp(r_time() + 1000000);
}

void
my_switching(struct proc *p, struct cpu *c)
{
    p->state = RUNNING;
    c->proc = p;
    swtch(&c->context, &p->context);
    c->proc = 0;
    if(p->state == RUNNABLE) my_queueing(p);
    release(&p->lock);
    acquire(&priority_boost_lock);
    if(priority_boost) priority_boosting();
    priority_boost = 0;
    release(&priority_boost_lock);
}

void
priority_boosting(){
    for(int i = 0; i < 3; i++) {
        acquire(&mlfq[i].lock);
        mlfq[i].s = mlfq[i].e = 0;
    }
    for(struct proc *p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        p->priority = 3;
        p->level = 0;
        p->time_quantum = 1;
        if(p->state == RUNNABLE){
            mlfq->p[mlfq->s++] = p;
        }
        release(&p->lock);
    }
    for(int i = 2; i >= 0; i--) release(&mlfq[i].lock);
}
```

system call

- void yield()

이미 구현돼 있어서 sys_yield로 wrapper를 만들어 system call에 등록했다.

- int fcfsmode()

mlfq, fcfs의 큐들을 전부 초기화하고 RUNNABLE인 process를 fcfs의 큐에 넣게 구현했다.

- int mlfqmode()

fcfs의 원소를 처음부터 보면서 level0큐인 mlfq[0]에 순서대로 넣었다.

- int getlev(), int setpriority(int pid, int priority)

명세서 대로 구현했다.

다음은 system call을 등록하는 과정이다.

```
#define SYS_yield 22
#define SYS_fcfsmode 23
#define SYS_mlfqmode 24
#define SYS_setpriority 25
#define SYS_getlev 26
```

구현한 system call들의 system call 번호

```
extern uint64 sys_yield(void);
extern uint64 sys_fcfsmode(void);
extern uint64 sys_mlfqmode(void);
extern uint64 sys_getlev(void);
extern uint64 sys_setpriority(void);
```

외부에서 선언한 system call 함수들의 외부 선언이다.

```
[SYS_yield] sys_yield,
[SYS_fcfsmode] sys_fcfsmode,
[SYS_mlfqmode] sys_mlfqmode,
[SYS_setpriority] sys_setpriority,
[SYS_getlev] sys_getlev
```

syscall table에 system call을 등록한다.

```
void yield(void);
int fcfsmode(void);
int mlfqmode(void);
int getlev(void);
int setpriority(int, int);
```

user.h에 선언한 system call

```
entry("yield");
entry("fcfsmode");
entry("mlfqmode");
entry("setpriority");
entry("getlev");
```

usys.pl에 등록한다. user.h에서 선언한 것과 같은 이름으로 한다.

번호 순으로 등록해야 한다.

implementation detail

- yield를 호출했을 경우에도 time quantum을 소모하게 했다. 이것이 좀더 일관성 있는 구현을 만든다.
- tick 초기화는 tick을 0으로 하고 100ms뒤 타이머 interrupt가 발생하게 했다.
- 스케줄러 및 system call들은 my_proc.c에 구현해 놓았다.
- main함수에서는 scheduler(void) 대신 my_scheduler(void)를 호출하게 했다.
- mlfq 큐

i-level 큐는 mlfq[i]로 관리한다.

- lock 관련
 - scheduler가 process를 queue에서 하나 뽑았을 때는 아직 해당 process에 대한 lock을 잡고 있다. my_switching함수에서 process lock을 해제한다.
 - schedule_mode에 접근, 수정할 경우 항상 lock을 잡게 했다.
 - queue에 대해서도 접근, 수정시 항상 lock을 잡게 했다.

- setpriority

- sys_setpriority는 두개의 argument를 argint함수로 받아오게 했다.

```
argint(0, &pid);  
argint(1, &priority);
```

- issue

FCFS 구현을 처음에는 timer interrupt를 꺼버리는 방식으로 구현했다. 이 방식은 xv6의 정상적인 booting을 방해하고 user 키보드 interrupt를 줘야 정상적으로 shell 화면이 출력됐다. 이를 해결하기 위해 timer interrupt를 끄는 대신 trap.c에서 timer interrupt가 발생했을 때 FCFS면 yield하지 않게 했다.

- results

실행결과는 부록에서 볼 수 있다.

- tests 결과

- FCFS

process가 순서대로 실행된 것을 볼 수 있다. example과 같은 결과를 보였다.

- mode switch

모드 스위치는 논리적으로 알맞은 결과이다. 처음에는 fcfs였으므로 0을 반환해서 안되고 mlfq로 바뀔 때는 정상적으로 바뀌어야한다. example과 같은 결과를 보였다.

- MLFQ

L0와 L1, L2에서 실행되는 횟수는 정확히 조절할 수는 없지만 $L0 < L1 < L2$ 순으로 횟수 개수가 커지는 경향을 보여야 한다. priority boosting은 tick=50돼야 생기고 각 프로세스는 $50/4=12.5$ tick 씩 갖는 것을 알 수 있다. L0는 1, L1은 3, L2는 8.5 tick 동안 실행된다는 것을 대략적으로 알 수 있다. 프로세스 별로 L0, L1, L2의 실행 비를 나타내면 1:3.3:6.1, 1:3.0:5.5, 1:3.1:5.6, 1:2.9:5.4로 나타난 것을 알 수 있다. L0, L1은 계산값과 거의 맞아 떨어지지만 L2에 대해서는 다른데 그 이유로 알 수 있는 것은 L2 queue에서 pop하는 과정이 L1, L0보다 오버헤드가 큰 것이 반영된 결과이다.

- my_tests

my_tests는 따로 정의한 테스트 구현으로 test1, test2, test3, test4, test5가 내부적으로 있다. test1에서는 fcfs, test2에서는 mlfq, test3에서는 mode switch, test4에서는 setpriority, test5에서는 yield가 정확히 구현된지 확인한다.

- test1

0->1->2->3 순으로 출력되어야 fcfs가 올바르게 구현된 것이다. result에서는 이를 확인할 수 있다.

- test2

mlfq에서 round robin이 정상적으로 되는지 검증한다. process들이 번갈아 실행되는지를 확인한다. 모든 레벨에서 라운드 로빈으로 출력된 것을 알 수 있다. level=2 queue에서는 가장 priority가 높은 것 중에 가장 먼저 큐에 들어온 것을 pop하게 구현했기에 라운드 로빈이 되는 것이 옳바르다.

- test3

fcfs와 mlfq 전환이 정상적으로 구현됐는지 확인한다. k가 1씩 증가하면서 100000의 배수가 되면 mlfq가 되고 k가 50000의 배수가 되면 fcfs가 된다. 그리고 k가 10000의 배수마다 번호를 출력한다. k가 50000의 배수가 되면 번호가 연속적이고 100000의 배수가 되면 번갈아 가면서 나오는지 확인하면 된다. 테스트 결과 50000의 배수가 등장하고 $(k \bmod 100000) = 90000$ 이 출력되기 전까지 한 process에서만 출력이 되는 것을 알 수 있다. 이후에는 번갈아 출력되는 것을 볼 수 있다.

- test4

mlfq 모드에서 i번 프로세스는 priority를 i로 계속 설정한다.

그리고 getlev로 queue level이 2일 때만 출력하게 한다. 최대한 fcfs와 결과가 유사한지 확인한다.

이는 i가 클수록 최대한 앞에 출력된다. 이는 setpriority와 L2큐가 정상적으로 구현됐음을 보여준다

- test 5

0, 1이 번갈아가며 출력되는지 확인한다. 또한 yield 이후 time quantum을 소모하는지 확인하기 위해 getlev로 i=0 프로세스와 i=1 프로세스의 level을 출력한다. 0 1이 번갈아가며 출력됐고 각 L0, L1큐의 time quantum만큼 소모된 것을 알 수 있다. 또한 큐의 level순으로 출력된 것을 알 수 있다.

Conclusion

테스트 결과 구현이 성공적이었음을 알 수 있었다.

appendix

test

FCFS & MLFQ test start

[Test 1] FCFS Queue Execution Order

Process 4 executed 100000 times

Process 5 executed 100000 times

Process 6 executed 100000 times

Process 7 executed 100000 times

[Test 1] FCFS Test Finished

nothing has been changed

successfully changed to MLFQ mode!

[Test 2] MLFQ Scheduling

Process 8 (MLFQ L0-L2 hit count):

L0: 9600

L1: 31915

L2: 58485

Process 9 (MLFQ L0-L2 hit count):

L0: 10504

L1: 31257

L2: 58239

Process 10 (MLFQ L0-L2 hit count):

L0: 10261

L1: 31924

L2: 57815

Process 11 (MLFQ L0-L2 hit count):

L0: 10674

L1: 31450

L2: 57876

[Test 2] MLFQ Test Finished

xv6 재부팅 후 my_tests 실행

my_tests

-----test1 start-----

0

0

0

1

1

1

2

2

2

3

3

3

4

4

4

-----test2 start-----

0: 0

1: 0

2: 0

3: 0

4: 0

0: 1

1: 1

2: 1

3: 1

4: 1

0: 2

1: 2

2: 2

3: 2

4: 2

0: 2

1: 2

2: 2

3: 2

4: 2

0: 0

1: 0

2: 0

3: 0

4: 0

0: 1

1: 1

2: 1

3: 1

4: 1

0: 2

1: 2

2: 2

3: 2

4: 2

fin 0

fin 2

fin 1

fin 4

3: 0

fin 3

-----test3 start-----

0: 0

1: 0

2: 0

2: 10000

0: 10000
1: 10000
0: 20000
1: 20000
2: 20000
0: 30000
1: 30000
2: 30000
0: 40000
1: 40000
2: 40000
0: 50000
0: 60000
0: 70000
0: 80000
0: 90000
0: 100000
1: 50000
1: 60000
1: 70000
1: 80000
1: 90000
1: 100000
2: 50000
2: 60000

2: 70000

2: 80000

2: 90000

2: 100000

0: 110000

1: 110000

2: 110000

0: 120000

1: 120000

2: 120000

0: 130000

1: 130000

2: 130000

0: 140000

1: 140000

2: 140000

0: 150000

0: 160000

0: 170000

0: 180000

0: 190000

0: 200000

1: 150000

1: 160000

1: 170000

1: 180000

1: 190000

1: 200000

0: 210000

fin 0

2: 150000

2: 160000

2: 170000

2: 180000

2: 190000

2: 200000

1: 210000

fin 1

2: 210000

fin 2

-----test4 start-----

3

3

3

2

3

3

fin 3

2

2

1

2

2

fin 2

1

1

0

1

1

fin 1

0

0

0

0

fin 0

-----test5 start-----

i: level

0: 0

1: 0

0: 1

1: 1

0: 1

1: 1

0: 1

1: 1

0: 2

1: 2